Middle East Technical University **Department of Computer Engineering**

CENG444 - Language Processors

2025-2026 Spring
Project III
Semantic Analysis & Introductory IR

Problem

In this assignment, you are required to write a C++ program that performs syntactic and semantic conformance tests for an experimental language called as *rowset manipulation language (RML)*. Your solution will implement partial implementation of graphical representation of semantics, which is essential part of intermediate representation (IR).

RML

An RML module consists of a series of "select" statements. A valid RML statement conforms to a set of syntactic rules that are mostly similar to the rules of the conventional expressions we are familiar with. The type system of RML is based on primitive types, which are string, number, and Boolean and rowset. RML does not have flow control structures like loops, decisions, and compound statements. However, implementation of a "select" statement involves implicit loops and conditional execution components without explicit constructs that we are familiar from imperative languages.

RML manipulates CSV files each having a header row and zero or more data rows. You do not need to worry about processing details of a CSV file because a supplemental library will be provided to help you focus on essence of language processing. **Additionally, this particular assignment is solely about syntax checking, you will not need to use and apply CSV processing techniques.**

Types

Since the `rowset` type can be reduced to other basic types at runtime, RML can be said to be a dynamically typed language. This does not mean that compile time checking (static type checking) is completely deferred. There is no explicit variable declaration construct in RML to define a variable. Instead, variable types are received from the CSV files. Any column defined in a CSV file becomes a variable that may receive partially or full qualified references. Once a variable is declared, its type cannot be changed.

Below is the list of the types allowed in RML.

Type	Description
`number`	Type to represent IEEE 754 double precision floating point value.
`boolean`	Type to represent boolean values.
`string`	String of characters.

Type	Description
`rowset`	An object that encapsulates rows each of which is composed of columns. When a rowset has a single row and a single column, it can be reduced to number, Boolean, or string depending on the column type.

Select Statements

"Select statements" is the only kind of statements that can be found in a valid RML program. An RML program must contain at least one select statement. A "select statement" is formed by placing a semicolon after a "select operation".

The "select statement" is the major element of an RML module. Each select statement contains a select operation, which is a ternary operator. The operands are named as "column list", "product space", and "condition".

Column List: A column list is a list of <expression, column alias> couples noted as noted below

```
<expression> as <column-alias>
```

This helps define the identifiers of the resulting *rowset* columns. A column alias has the syntax of an identifier.

Product Space: Syntactically, a product space is a comma separated list of rowsets implying the cross product of the spaces to form the final rowset that will be iterated over for calculation. A rowset appearing in the list is called as a **space**. A space can be declared in two ways: External or calculated. An external space is a single identifier that references to a csv file. In such a case, the file is located by using the filename synthesized as "<identifier>.csv". A calculated space is a select statement enclosed in parentheses. Any space may have a space-alias that makes it possible to arrange unambiguous rowset references from within the expressions.

Condition: A Boolean value evaluating expression that is used to qualify each row in the product space. A condition must not involve top level comma operator which leads production of more than one value.

Example:

```
select A.name as name, A.surname as surname, B.grade as grade

   from (select P.id as id, P.grade as grade from student P where P.typ=="external") A,
coursegrading B

   where B.studentid==A.id && B.grade>=70;

select max(grade) as maxGrade from coursegrading where coursecode=="CENG444";

select A.name as name, A.surname as surname from student A, coursegrading B

   where B.studentid==A.id &&  B.grade==(select max(grade) as maxGrade from coursegrading
where true);
```

In this example, B.grade is a fully qualified variable, and the "P" in "student P" is a space alias. You can spot more than one fully qualified variables and space aliases.

Expressions

The form of the expressions in RML is mostly conventional. This section defines the entities that can be used to construct a valid RML expression.

- Operators

  RML has a rich set of operators with various number of operands and associativities. You are accustomed to many of them from your earlier experience. See Appendix 1: OperatorsAppendix 1: Operators.

- Parentheses

  An expression can be enclosed in parentheses to change/ensure evaluation order. See the precedence and associativity of the operators to understand the default evaluation order.

  Additionally, any rowset, which is a first class citizen in RML, must be a "select" enclosed in parentheses.

- String Literals

  A string is in the form of regular and exceptional characters enclosed in double quotes. Exceptional characters are double quote("), and back-slash(\), and new-line(character code 10).

Exceptional Character	**Encoding in string**
double quote(")	\"
back-slash(\)	\\
new-line	\n

  A character, regardless if it is exceptional or regular, can be coded with help of byte coding. Any byte except zero can be encoded in a string of characters by using the hexadecimal form, which is \xhh, where h is a hexadecimal digit. Capital and small letters can be used in hexadecimal digit specification.

- Numeric Literals

  The numeric literals can be composed of the valid combinations of whole part (W), decimal separator (.), and fractional part (F). Whole part can be either zero, or a sequence of digits starting with a non-zero digit. Fractional part is a nonempty sequence of digits, not ending with zero. Following are valid examples of W,".", and F combinations:

W	**.**	**F**	**Example**
1	1	1	1.25
1	1	1	0.25432
0	1	1	.25
0	1	1	.0134
1	0	0	123
1	0	0	0

  A number can have an optional exponential part prefixed by 'e' or 'E'. When supplied, an optional sign that can be either '+' or '-' may follow. Then, a sequence of digits must be given. The first digit must be non-zero.

- Boolean Literals

  `true` and **`false`** are the boolean literals.

- Identifiers

  An identifier starts with a letter or an underscore. The remaining characters in an identifier can be letters, underscores, and decimal digits. RML identifiers are case sensitive so `var` and `Var` are two distinct variables.

- Function Calls

  A function call is formed as <identifier> followed an expression list enclosed by parentheses. The list of expressions is skipped for the functions having no formal parameters.

White Spaces

The tokens found in a valid RML code must be separated by at least one white space character, or special character such those that can be found in operators. The developer is free to use as many white spaces as required to improve readability of the code.

RML Program

A valid RML program is a sequential list of statements that satisfy following conditions:

1. Statements must be syntactically correct.
2. Statements must be free of semantic errors.

Your Assignment

You are required to develop a C++ program that accepts one command line argument as the text file to be compiled to achieve semantic representation, which implements partially the intermediate representation that creates the basis for final translation. Your program will generate an output file having JSON format. The name of the output file will be based on the name of the input file. The extension of the input is assumed as `.txt`, so the input file name will be specified without extension. The output file will have `.json` extension.

Example run:

When your program is compiled and run as "`project3 test1`", your program will read the input file "`test1.txt`", and generate the output as "`test1.json`". In case your program fails to open the input file, your program will print "File not found!" message and terminate.

Processor Output

The processor you develop (`project3`) will accept the name of the source file that will be compiled. The program must contain only one parameter. No parameter or more than one parameter cases will be reported as errors. The processor must create an output file that conveys the symbol table, the list of the statements with augmented expression trees, and the messages, which are mainly the result of the syntactic and semantic checks.

The results generated by the processor you are required to develop must fully conform to the format that can be observed by the supplied samples.

Detected Symbols

The language processor is required to generate the symbol table of the identifiers defined by the statements. The symbol table will have entries with space reference, symbol name and type.

Expression Trees with Types

The language processor that you will develop is required to generate tree of expressions. Each node in the expression tree must have a type calculated during compile time. The type of the root node of the expression becomes the type of the expression.

Semantic Properties and Constraints

1. Each select operator establishes a symbol table that ensures uniqueness of symbols that can be referenced from the expressions that are components of the select operator. This implies hierarchy of symbol tables that will be used in resolution of column references. The resolution tries to find the column reference by using the innermost (the lowest level) symbol table within a select operator.
2. A column in a symbol table cannot be redefined.
3. An aggregate function can appear only at the root node of a column definition of a select operator.
4. An aggregate function cannot be operand of any operator.
5. A column definition cannot have comma operator.
6. If a column in a column list is defined as an aggregate, all other columns must also be defined as aggregate.
7. Any runtime library function name cannot be redefined as a space name, as a column name, or an alias.
8. The first operand of a ternary operator can be a Boolean only.
9. The Boolean operators (and, or) are applied to booleans only.
10. Multiplication, subtraction, and division operators are applied to numbers only.
11. The + operator can be used in various configurations.
    a. `<number> + <number>` generates a `<number>`.
    b. `<string> + <string>` generates a `<string>`.
    c. `<string> + <number>` generates a `<string>`.
    d. `<number> + <string>` generates a `<string>`.
12. A not (`!`) operator is applied on a boolean value only.
13. A unary minus (–) operator is applied on a numeric value only.
14. A parameter list (actual parameters) passed to a function must conform to the formal parameters. This covers the number of parameters and the type equality of respective parameters.
15. The first operand of a call operator (callee) can be an identifier only.
16. A function call without any parameter is possible.
17. Any operator that is not applicable to the operand(s) found is an error in the code.
18. Seeing the attached table of operators, for the operators having two operands of type1 and type2, type1 must be identical to the type2.
19. The type *rowset* cannot be operand of any operator given in the attached table of operators.
20. A select operation results in type of its first column if its column list contains only one column. Otherwise, its type must be set as *rowset*.

Regulations & Hints

- **Implementation:** You should use Flex, Bison with C++ to develop your program. Make sure that your program will accept the name of the input file. In case the program is run without a parameter or with more than one parameter your program must terminate immediately with appropriate error message sent to the standard output.
- **Supplemental Material:** You will be provided with a supplemental code and an example set for support and standardization.
    - A standardized grammar for RML
    - A small library for fundamental data structures, runtime functions, and the language runtime.

  Note that you may be asked to develop your own extensions to any of these components.
- **Evaluation:** The evaluation will be based on correctness of each component (the symbol table, the expression tree, and the semantic check list).
- **Submission:** You need to submit all relevant files you have implemented (`.cpp, .h, .l, .y`, etc.) as well as a `README` file with instructions on how to build and run, in a single `.zip` file named `<your studentID>`.

Following notes are for more clarity to address possible concerns and/or questions:

- Use Flex and Bison to generate C++ not C file. C implementation will not be accepted.
- Never modify the automatically generated files generated by Flex and Bison. You may prefer consulting the recitation material and the sample project on semantic analysis.
- Describe precisely the steps to build and run your program in `README.txt` file, which is essential part of your submission. Provide any configuration items such as scripts, configuration files that will be necessary.

Appendix 1: Operators

Below are the operators that can be used in RML expressions. For each operator, form (coding syntax), possible applications on types, type of the result, associativity, and precedence rules are given.

OP	Short Name	Form	Application	Result Type	Assoc.	Prec.
,	Comma	Binary		Multi	LR	0
?:	Conditional	Ternary	<boolean>?<type1>:<type2>	<type1>		1
&&	Boolean and	Binary, infix	<boolean> && <bolean>	<boolean>	LR	2
||	Boolean or	Binary, infix	<boolean> || <bolean>	<boolean>	LR	2
==	Equal	Binary, infix	<type1> == <type2>	<boolean>	LR	3
!=	Not equal	Binary, infix	<type1> != <type2>	<boolean>	LR	3
<	Less than	Binary, infix	<type1> > <type2>	<boolean>	LR	3
>	Greater than	Binary, infix	<type1> < <type2>	<boolean>	LR	3
<=	Less than or equal	Binary, infix	<type1> <= <type2>	<boolean>	LR	3
>=	Greater than or equal	Binary, infix	<type1> >= <type2>	<boolean>	LR	3
+	Add	Binary, infix	<number> + <number>	<number>	LR	4
+	Concatenate	Binary, infix	<string> + <string>	<string>	LR	4
+	Concatenate	Binary, infix	<string> + <number>	<string>	LR	4
+	Concatenate	Binary, infix	<number> + <string>	<string>	LR	4
-	Subtract	Binary, infix	<number> - <number>	<number>	LR	4
*	Multiply	Binary, infix	<number> * <number>	<number>	LR	5
/	Divide	Binary, infix	<number> / <number>	<number>	LR	5
-	Negate	Unary, prefix	-<number>	<number>	RL	6
!	Boolean negate	Unary, prefix	!<boolean>	<boolean>	RL	6
()	Call function	Binary	id(<expression >)	Return type of function	LR	7

Appendix 2: RML Runtime Library

RML runtime library includes a limited set of functions that can be accessed by using function call operator. RML, has two types of functions:

- Aggregate functions (A):

  These functions can be used in column lists only. An aggregate function in a column, calculates a single value by using each eligible column found in the *rowset* generated by the select operator which the column belongs to.

- Scalar functions (S):

  A scalar function generates does not have limitation of aggregate functions. It operates on its parameters and returns a value.

Index	Prototype	A/S	Description
0	`number stddev(numbers)`	A	Calculates standard deviation of the values calculated for each of the rows
1	`number mean(numbers)`	A	Calculates the mean of the values calculated for each of the rows.
2	`number count()`	A	Calculates row count.
3	`number min(numbers)`	A	Calculates the minimum of the values calculated for each of the rows.
4	`number max(numbers)`	A	Calculates the maximum of the values calculated for each of the rows.
5	`number sum(numbers)`	A	Calculates the sum of the values calculated for each of the rows.
6	`number sin(number)`	S	Calculates the sine of a given number.
7	`number cos(number)`	S	Calculates the cosine of a given number.
8	`number tan(number)`	S	Calculates the tangent of a given number.
9	`number pi()`	S	Returns pi.
10	`number atan(number)`	S	Calculates the inverse tangent of a given number.
11	`number asin(number)`	S	Calculates the inverse sine of a given number.
12	`number acos(number)`	S	Calculates the inverse cosine of a given number.
13	`number exp(number)`	S	Evaluates e powered to the given parameter.
14	`number ln(number)`	S	Calculates natural logarithm of the given parameter.
15	`number number(string)`	S	Converts a string to the corresponding number.
16	`number random(number)`	S	Returns a random number x for a given number y (y>0) with the following condition: 0 <= x < y.

Index	Prototype	A/S	Description
17	`number len(string)`	S	Returns the length of a given string.
18	`string right(string, n)`	S	Returns a new string containing rightmost n characters of the first parameter.
19	`string left(string, n)`	S	Returns a new string containing leftmost n characters of the first parameter.

Appendix 3: Sample RML Program

Below is an error free sample RML program. Note that the first set of the supplemental material will convey sample external spaces (csv files), and the result JSON file that is expected after processing these samples.

Sample 1: Syntactically and semantically correct

```
select E.ABC as col1 from (select sid as ABC from students where true)
E, courses F where F.cid=="726";
```

Sample 2: Syntactically and semantically correct

```
select B.firstname as name, B.lastname as surname, A.grade as grade
   from (select P.sid as id, P.grade as grade from grades P where
P.cid=="C01") A, students B
   where B.sid==A.id && number(A.grade)>=92;
select max(number(grade)) as maxGrade from grades where cid=="C07";
select A.firstname as name, A.lastname as surname from students A,
grades B, courses C
   where B.sid==A.sid &&  number(B.grade)==(select max(number(grade))
as maxGrade from grades where true)
          && C.cid==B.cid;
```